Tabletec
Restaurant Booking and Meal Preorder System



Prof. Valon Raca

SWEN-383 Software Design Principles and Patterns

Amar Ismaili, Fabian Berisha, Yll Morina

CIT Students at RIT-K

26 October 2026

Rochester Institute of Technology in Kosovo

1. Project Description

Overview

Tabletec is a booking system for restaurant websites. Guests reserve a table and can order their meals in advance. We use an example Italian restaurant, Brace e Vino, to design and test it.

Problem

A table can look free to two guests at the same time, and the kitchen only has so many portions per seating. If the table and the food are booked separately, one can succeed while the other fails. Tabletec checks both together, so a booking is either fully confirmed or not saved at all.

Users

Guests check availability, pick meals if they want, book, and cancel before the visit.
Staff see the bookings, turn menu items on or off, and mark guests as seated or finished.

How it works

The guest picks a date, party size and time, adds meals, and enters a name and email. The system checks the table and the portions again, saves the booking and shows a reference number with a private key for managing it.

2. Scope

Features

F1 See which seatings are free for a date and party size.
F2 Book a table.
F3 Preorder meals (optional).
F4 Look up or cancel a booking with its reference and key.
F5 Staff view bookings and manage the menu.

Rules

One restaurant with five tables: two for 2 people, two for 4 and one for 6. Parties of 1–6 get one table. Three seatings a night: 18:00, 19:30 and 21:00, each 90 minutes. Bookings can be made up to 30 days ahead, and meal orders close one hour before the seating. Prices are saved with the booking, so later menu changes do not affect it.

Booking status

A booking goes from confirmed to seated to completed, or from confirmed to cancelled. Cancelling frees the table and the portions.

Not included

Online payment, delivery, waiting lists, joining tables, more than one restaurant, and real emails.

3. Class Diagram: Domain

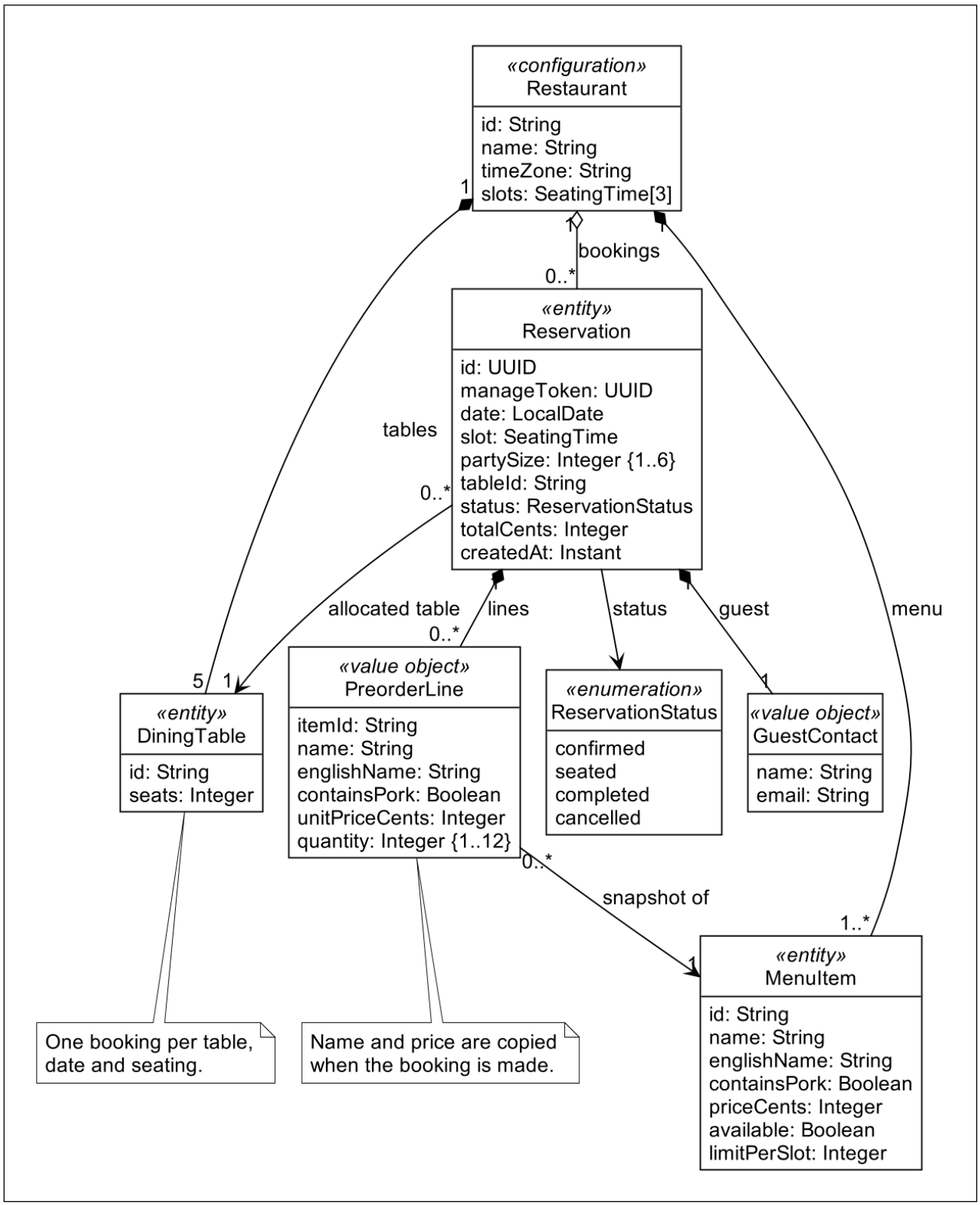


Figure 1. The main data. A reservation keeps its own copy of the guest details and ordered meals.

4. Class Diagram: Application

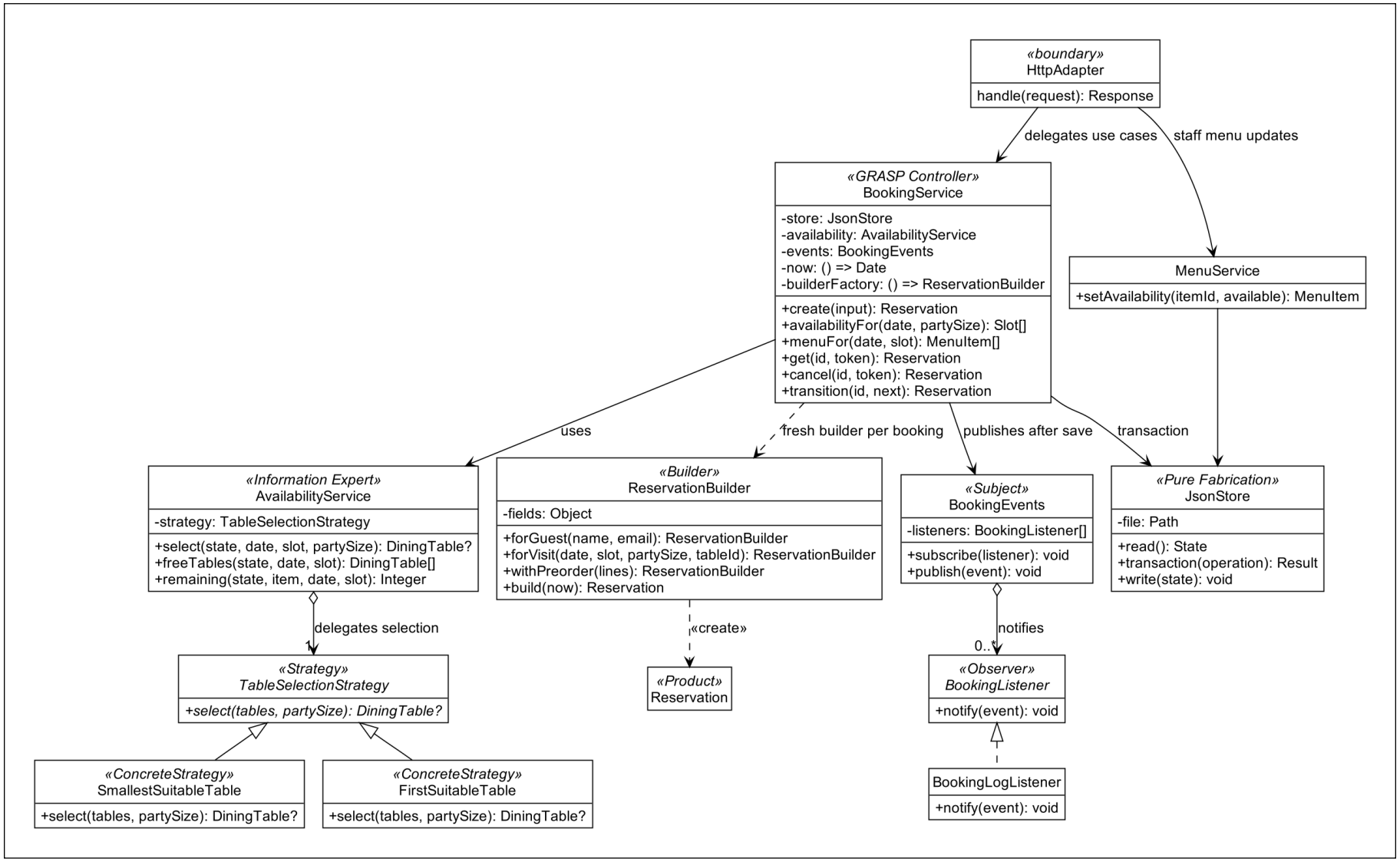


Figure 2. The main classes and where the three planned patterns fit (Strategy, Builder, Observer).

5. Sequence Diagram: Making a Booking

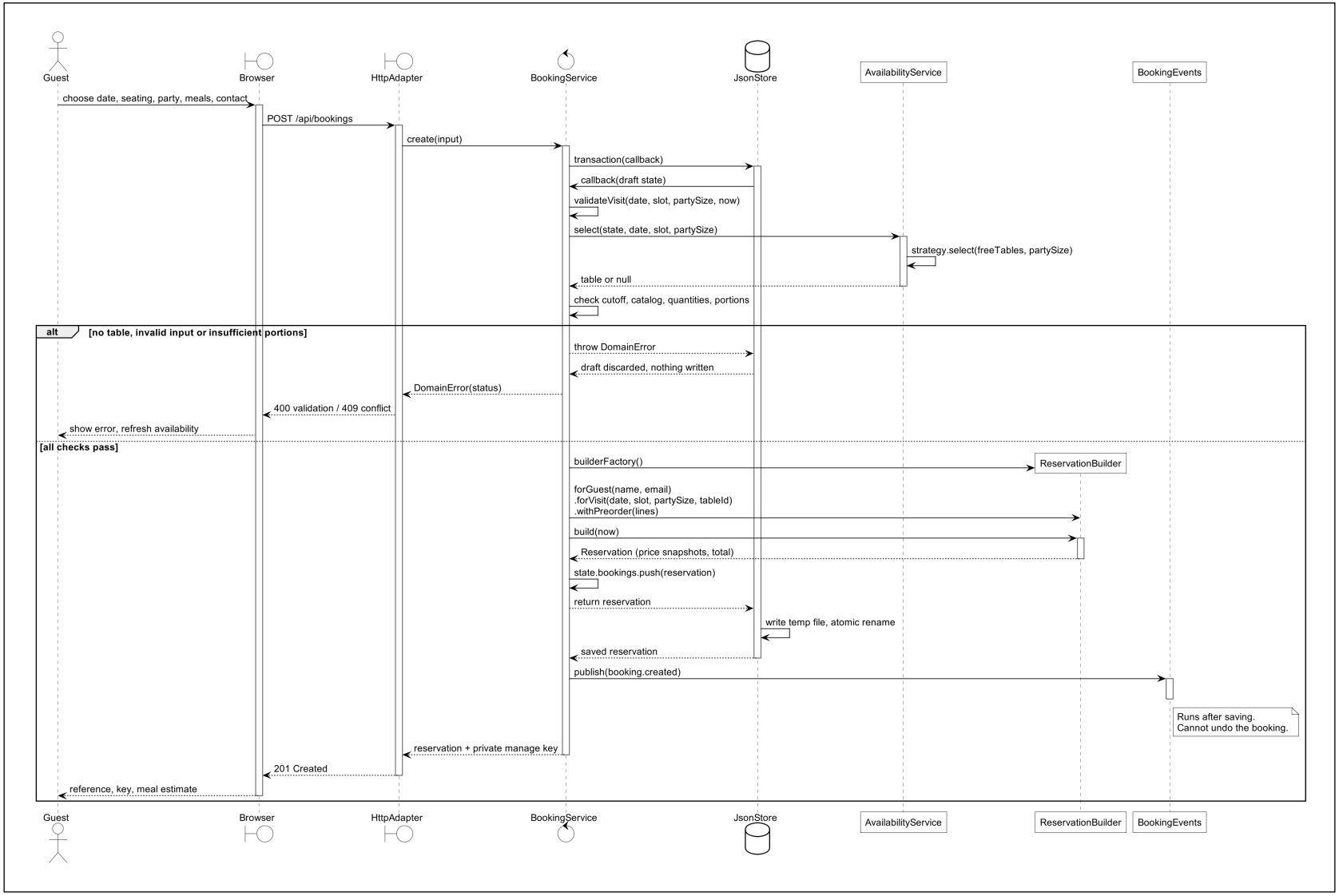


Figure 3. Booking a table with meals. If any check fails nothing is saved; otherwise the booking is saved and then listeners are notified.

6. Design Principles

SOLID

Single responsibility: each class has one job. BookingService handles bookings, MenuService the menu, JsonStore saving data.

Open/closed: a new table-choosing rule can be added as a new strategy without changing BookingService.

Liskov: every strategy takes the free tables and party size and returns a table that fits, or none.

Dependency inversion: BookingService gets its store, strategy and event publisher passed in, so they can be swapped or faked in tests.

GRASP

Controller: BookingService receives the booking requests.

Information Expert: AvailabilityService knows which tables and portions are free.

Creator: ReservationBuilder creates reservations.

Pure Fabrication: JsonStore keeps saving data separate from the booking rules.

Low coupling, high cohesion: classes talk through small interfaces and each stays focused.

7. Design Risks and Planned Patterns

Risk 1: Double bookings and changing table rules *(Strategy)*

Risk: two guests try to book the last table at once, and the way we pick tables may change later.

Plan: table choice is a Strategy (SmallestSuitableTable, FirstSuitableTable), so rules can be swapped. The booking itself is checked and saved in one transaction, which is what stops double bookings.

Trade-off: a few extra classes for a small rule.

Risk 2: Half-finished bookings *(Builder)*

Risk: meals are optional, so a booking could be saved with missing or wrong details.

Plan: a Builder puts the guest, visit and meals together into one complete Reservation, and nothing is saved until everything is valid.

Trade-off: more code than a simple constructor; a factory may be enough if bookings stay simple.

Risk 3: Extra features breaking bookings *(Observer)*

Risk: later features like kitchen alerts or emails could get mixed into the booking code, and their errors could break a booking.

Plan: with Observer, BookingEvents notifies listeners only after the booking is saved. A failing listener cannot undo it.

Trade-off: events are kept in memory, so they can be lost if the app crashes.

8. Summary

The table below links each risk to the pattern we plan to use.

Risk	Pattern	Classes
Double bookings, changing table rules	Strategy	TableSelectionStrategy, SmallestSuitableTable, FirstSuitableTable
Half-finished bookings	Builder	ReservationBuilder, Reservation
Extra features breaking bookings	Observer	BookingEvents, BookingListener

Next steps (Part 2)

Implement and test the three patterns, refine the diagrams, and write the final report.

Repository

The GitHub link is in the myCourses submission comment.

References

Freeman and Robson, *Head First Design Patterns*, 2nd ed., 2020.
Gamma, Helm, Johnson and Vlissides, *Design Patterns*, 1994.
Larman, *Applying UML and Patterns*, 3rd ed., 2004.

Use of AI tools

As allowed by the course policy, AI was used only for generalization and polishing. All ideas and the core mental models were designed by us, and AI was used only as a harness. The design decisions are our own and we can explain them.